

Inserimento nei B-tree

BtreeInsert(T, t, k) { % k non è già in T

$z \leftarrow T.root$

 if ($z.m = 2t - 1$) {

$s \leftarrow \text{Allocate Node}(t)$

$s.m \leftarrow 0$

$s.leaf \leftarrow \text{FALSE}$

$s.c_1 \leftarrow z$

$T.root \leftarrow s$

 BtreeSplit($s, z, 1, t$)

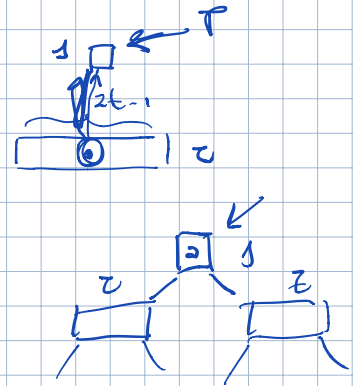
 BtreeInsert notFull(s, k, t)

 } else {

 BtreeInsert notFull(z, k, t)

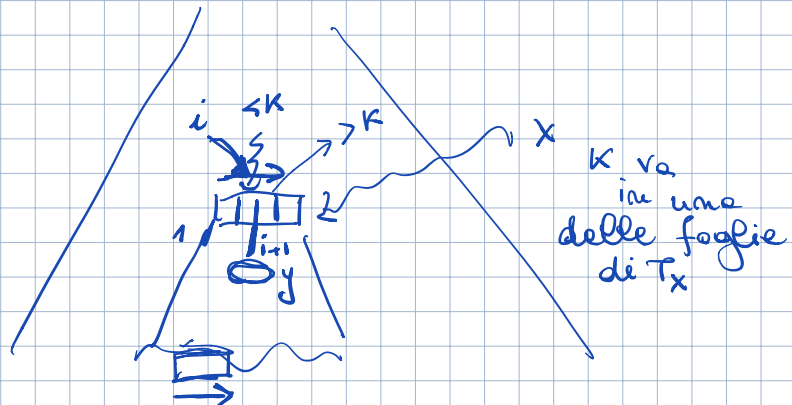
 }

}



x non è
pieno

x è già in
RAM



k va
in una
delle foglie
di T_x

BtreeInsertnotFull (x, k, t) { $\% x.m < 2t-1$
 $\% x \in \text{in RAH}$

if ($x.\text{leaf}$) {
 $i \leftarrow x.m$
 while ($i \geq 1 \ \&\& \ x.\text{key}_i > k$) {
 $x.\text{key}_{i+1} \leftarrow x.\text{key}_i$
 $i \leftarrow i-1$
 }

$x.\text{key}_{i+1} \leftarrow k$
 $x.m \leftarrow x.m + 1$

DiskWrite (x)

} else {
 $i \leftarrow x.m$
 while ($i \geq 0 \ \&\& \ x.\text{key}_i > k$) {
 $i \leftarrow i-1$
 }

$y \leftarrow \text{DiskRead}(x.c_{i+1})$

if ($y.m < 2t-1$) {
 BtreeInsertnotFull (y, k, t)
 }

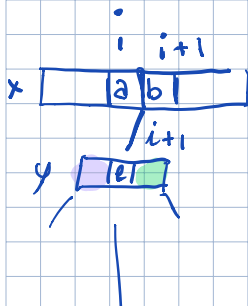
} else {

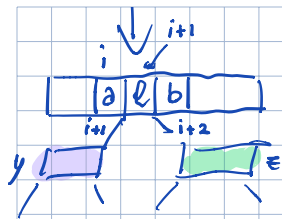
BtreeSplit ($x, y, i+1, t$)

if ($k < x.\text{key}_{i+1}$) {

$y \leftarrow \text{DiskRead}(x.c_{i+1})$

BtreeInsertnotFull (y, k, t)





```

} else {

```

$z \leftarrow \text{DiskRead}(x.c_{i+2})$

$\text{BtreeInsertNotFull}(z, k, t)$

```

}

```

```

}

```

```

}

```

```

}

```

Complexity $\left\{ \begin{array}{l} \text{CPU } O(t \cdot \log_t m) \\ \text{R/W } O(\log_t m) \end{array} \right.$

$m \cdot \infty$

Example

T vuoto

$t = 2$

1, 2, 3

Insertare

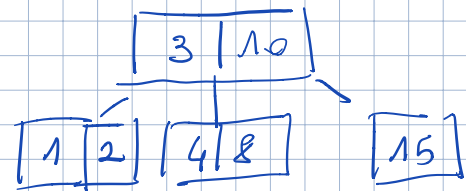
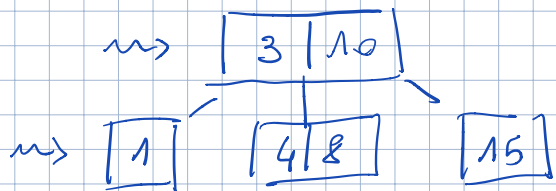
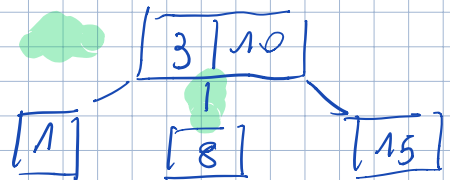
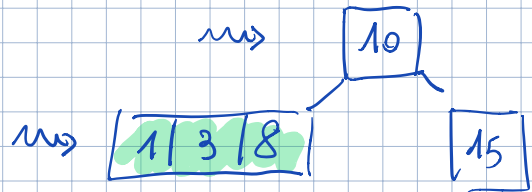
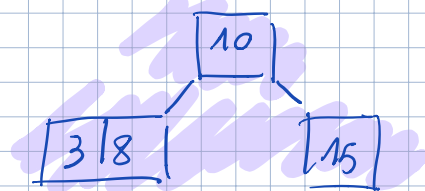
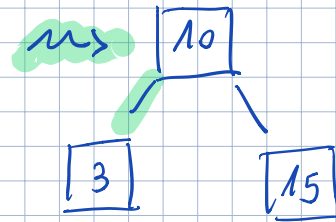
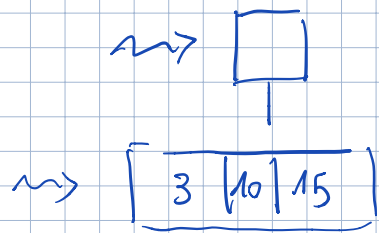
10, 15, 3, 8, 1, 4, 2, 12, 20

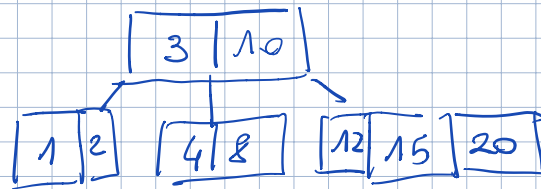
10

10 15

x.key
 3 10 15
 10 15

3 10 15



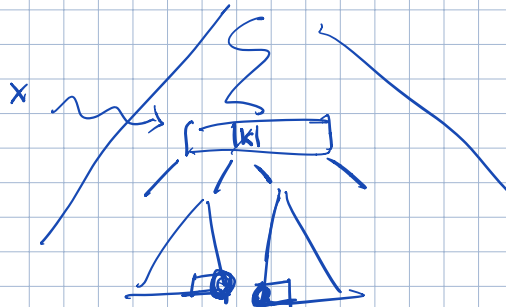


Btree Cancellazione

Intuitivamente

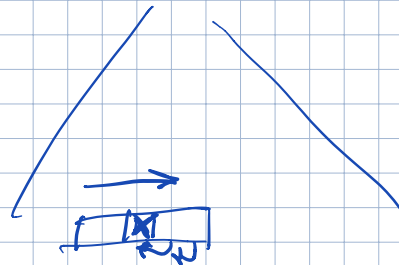
$T, K, x \leftarrow$ da trovare

- 1) Se K si trova in un nodo interno



Uso predecessore / successore come nei BST

- 2) Se K si trova in una foglia

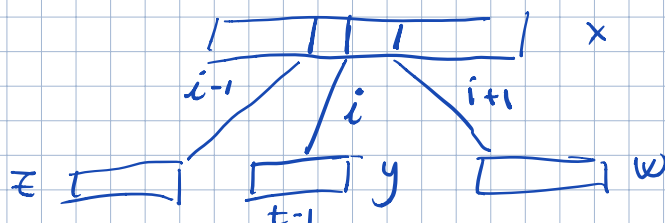


sposto a dx
le chiavi
maggiori di K

Attenzione Mentre scendo te incontro
un nodo con $t-1$ chiavi
(vuoto) cerca di aggiungergli
chiavi

⚡
"Inverso dello split"

Come aggiungere chiavi ad un nodo

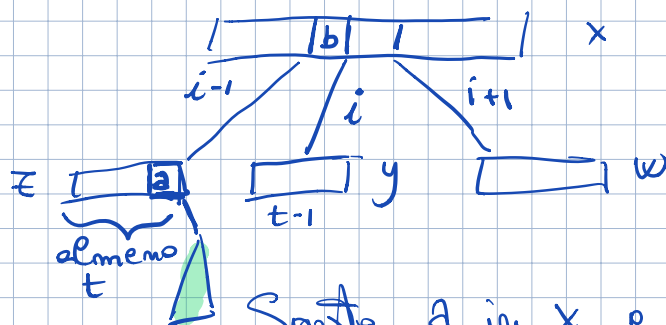


x ha sicuramente
almeno
 t chiavi

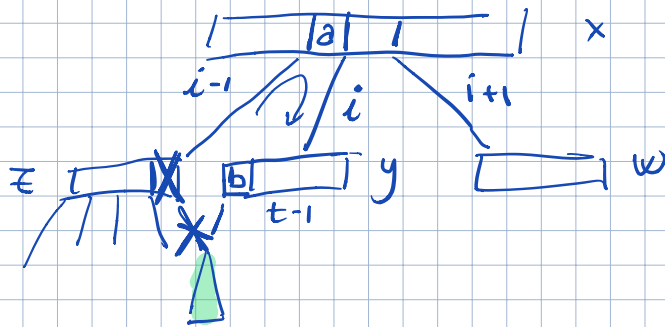
y ha solo $t-1$ chiavi

Controllo z e w

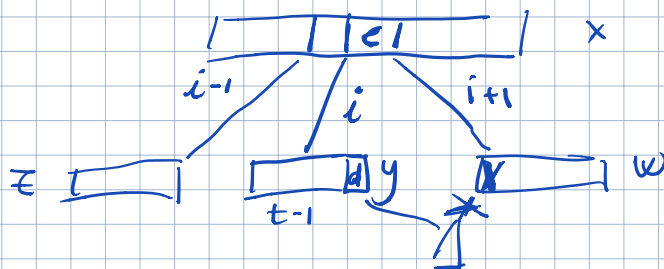
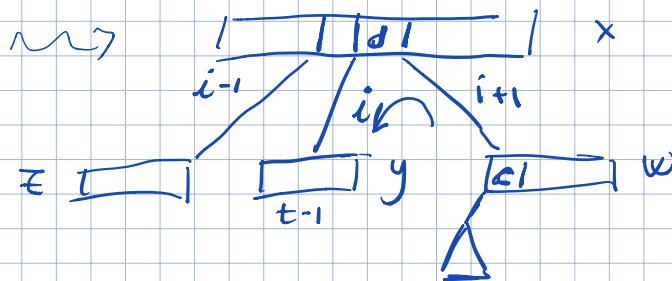
1) Se z ha più di $t-1$ chiavi
rubo una chiave a z



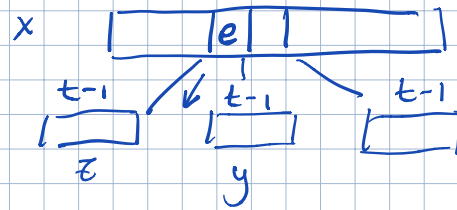
Sposto a in x e b in y
Sposto anche un figlio di z
in y



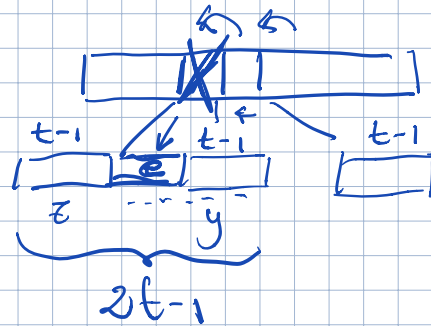
2) Se z ha solo $t-1$ chiavi, ma w ha più di $t-1$ chiavi



3) Se sia z che w hanno $t-1$ figli

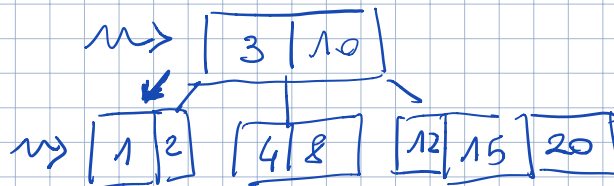


Biberge

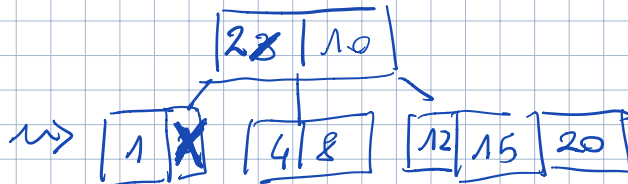


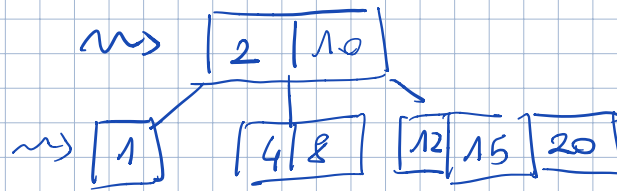
Esempio

$t=2$ $t-1=1$

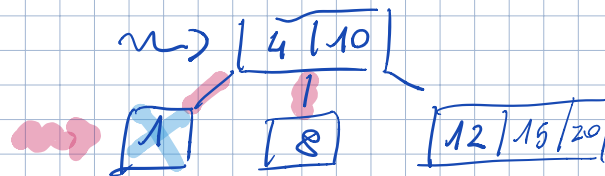
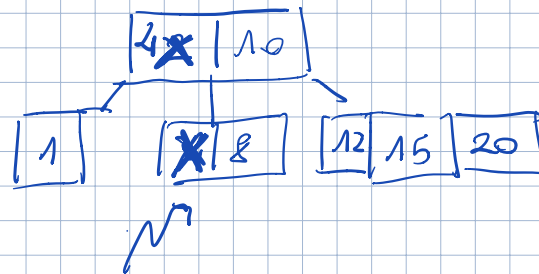


Cancellare 3

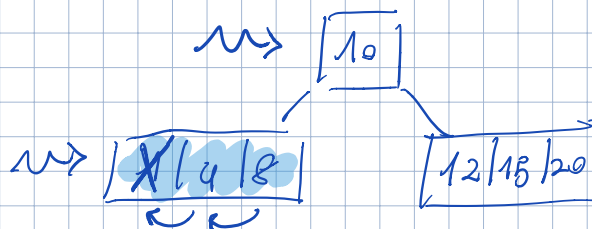




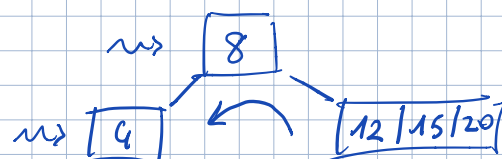
Cancellore 2



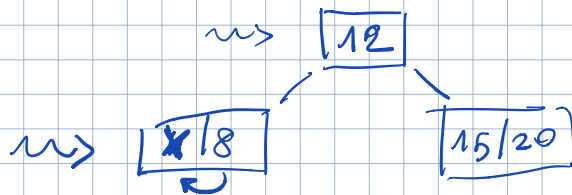
Cancellore 1



Cancellore 10



Cancellare k



$\sim \cdot \sim \cdot \sim$

Ad ogni passo sono su un nodo x
 x è in RAM, x ha almeno t chiavi
 k è da cancellare da T_x



1) Se x è una foglia e k è in x
 cancello k da x

2) Se x non è una foglia e k è in x

- Se il figlio z a sx di k ha almeno t chiavi $\leadsto$

Cancello da T_x il predecessore di k
 e lo scrivo al posto di k

- Se il figlio z a sx di k ha solo $t-1$ chiavi, ma il figlio w ha almeno t chiavi $\leadsto$

successore

- Se --- sia z che w hanno solo $t-1$ chiavi faccio il BMerge tra

$\mathbb{Z}$, W e la chiave K

Faccio la ch. ric. su questo nuovo nodo

3) Se x non è una foglia e K non è in x

Individuo il figlio y in cui scendere

- Se y ha almeno t chiavi scendo in y → ch. ric. su y

- Se y ha solo $t-1$ chiavi

- rubo una chiave al fratello z_x di y --- oppure

- rubo a z_x di y --- oppure

- BMerge tra y e il fratello z_x

Poi faccio la ch. ric.

Complessità $\left\{ \begin{array}{l} \text{CPU } O(t \cdot \log_t n) \\ \text{R/W}_{n \cdot t} O(\log_t n) \end{array} \right.$

